

编译原理proj4

周天羽菲 21305399

实验内容

优化实现

常量折叠

强度削弱

代数恒等式

死存储消除

公共子表达式消除

运行结果截图

评测机

本地

周天羽菲 21305399

实验内容

见ppt和doc

优化实现

常量折叠

说明

补充代码，实现下面这些二元运算的常量折叠。

做完性能没什么提升 1540 2024-06-18 22:48:45 1312 / 3600  

```
CMakeLists.txt  ConstantFolding.cpp  Instruction.def 9+ x  InstrTypes.h
opt > SYSU-lang > llvm > install > include > llvm > IR > Instruction.def > ...
142 HANDLE_UNARY_INST(12, FNeg , UnaryOperator)
143 LAST_UNARY_INST(12)
144
145 // Standard binary operators...
146 FIRST_BINARY_INST(13)
147 HANDLE_BINARY_INST(13, Add , BinaryOperator)
148 HANDLE_BINARY_INST(14, FAdd , BinaryOperator)
149 HANDLE_BINARY_INST(15, Sub , BinaryOperator)
150 HANDLE_BINARY_INST(16, FSub , BinaryOperator)
151 HANDLE_BINARY_INST(17, Mul , BinaryOperator)
152 HANDLE_BINARY_INST(18, FMul , BinaryOperator)
153 HANDLE_BINARY_INST(19, UDiv , BinaryOperator)
154 HANDLE_BINARY_INST(20, SDiv , BinaryOperator)
155 HANDLE_BINARY_INST(21, FDiv , BinaryOperator)
156 HANDLE_BINARY_INST(22, URem , BinaryOperator)
157 HANDLE_BINARY_INST(23, SRem , BinaryOperator)
158 HANDLE_BINARY_INST(24, FRem , BinaryOperator)
159
160 // Logical operators (integer operands)
161 HANDLE_BINARY_INST(25, Shl , BinaryOperator) // Shift left (logical)
162 HANDLE_BINARY_INST(26, LShr , BinaryOperator) // Shift right (logical)
163 HANDLE_BINARY_INST(27, AShr , BinaryOperator) // Shift right (arithmetic)
164 HANDLE_BINARY_INST(28, And , BinaryOperator)
165 HANDLE_BINARY_INST(29, Or , BinaryOperator)
166 HANDLE_BINARY_INST(30, Xor , BinaryOperator)
167 LAST_BINARY_INST(30)
168
```

强度削弱

说明

performance/dead-storage-elimination-1.sysu.c 分数升高（但没高多少）

1570 2024-06-19 17:38:52 1319 / 3600  

把乘法变成位运算

```
32 if (isPower2) {
33     if (TheInst->getOpcode() == Instruction::Mul) {
34         // 替换乘法指令的使用
35         TheInst->replaceAllUsesWith(BinaryOperator::CreateShl(
36             lhs,
37             ConstantInt::get(lhs->getType(),
38                               Log2_64(constRhs->getZExtValue())),
39             "shl.reduction",
40             TheInst));
41         toErase.push_back(TheInst);
42         ++reductionCount;
43     }
44 }
```

把除法变成位运算

```

44     else if (TheInst->getOpcode() == Instruction::SDiv) {
45         // 替换有符号除法指令的使用
46         TheInst->replaceAllUsesWith(BinaryOperator::CreateAShr(
47             lhs,
48             ConstantInt::get(lhs->getType(),
49                 Log2_64(constRhs->getZExtValue())),
50             "ashr.reduction",
51             TheInst));
52         toErase.push_back(TheInst);
53         ++reductionCount;
54     }
55     else if (TheInst->getOpcode() == Instruction::UDiv) {
56         // 替换无符号除法指令的使用
57         TheInst->replaceAllUsesWith(BinaryOperator::CreateLShr(
58             lhs,
59             ConstantInt::get(lhs->getType(),
60                 Log2_64(constRhs->getZExtValue())),
61             "lshr.reduction",
62             TheInst));
63         toErase.push_back(TheInst);
64         ++reductionCount;
65     }
66 }
67 }

```

把取余也进行优化

```

        ++reductionCount;
    } else if (TheInst->getOpcode() == Instruction::URem) {
        // 替换取余指令的使用
        IRBuilder<> builder(TheInst);
        Value* div = builder.CreateLShr(
            lhs,
            ConstantInt::get(lhs->getType(),
                Log2_64(constRhs->getZExtValue())),
            "div.reduction");
        Value* mul = builder.CreateMul(div, constRhs, "mul.reduction");
        Value* sub = builder.CreateSub(lhs, mul, "sub.reduction");
        TheInst->replaceAllUsesWith(sub);
        toErase.push_back(TheInst);
        ++reductionCount;
    }
}
// 删除被替换的指令

```

代数恒等式

说明

完成了加减乘除取余

[performance/algebraic-identity-1.sysu.c](#) 分数升高

```

ConstantFolding.cpp M  DSE.cpp U  AlgebraicIdentities.cpp U x  DSE.hpp U  ConstantFoldin
ask > 4 > AlgebraicIdentities.cpp > run(Module &, ModuleAnalysisManager &)
1  #include "AlgebraicIdentities.hpp"
2
3  using namespace llvm;
4
5  PreservedAnalyses
6  AlgebraicIdentities::run(Module& mod, ModuleAnalysisManager& mam)
7  {
8      int reductionCount = 0;
9
10     // 遍历所有函数
11     for (auto& func : mod) {
12         // 遍历每个基本块
13         for (auto& bb : func) {
14             std::vector<Instruction*> toErase;
15             // 遍历基本块的指令
16             for (auto& inst : bb) {
17                 if (auto binOp = dyn_cast<BinaryOperator>(&inst)) {
18                     // 获取操作数
19                     Value* lhs = binOp->getOperand(0);
20                     Value* rhs = binOp->getOperand(1);
21
22                     // 尝试将操作数转换为常整数
23                     auto constLhs = dyn_cast<ConstantInt>(lhs);
24                     auto constRhs = dyn_cast<ConstantInt>(rhs);
25
26                     switch (binOp->getOpcode()) {
27                         case Instruction::Add:
28                             //  $x + 0 = x$ 
29                             if (constRhs && constRhs->isZero()) {
30                                 binOp->replaceAllUsesWith(lhs);
31                                 toErase.push_back(binOp);
32                                 ++reductionCount;
33                             } else if (constLhs && constLhs->isZero()) {
34                                 binOp->replaceAllUsesWith(rhs);
35                                 toErase.push_back(binOp);
36                                 ++reductionCount;
37                             }
38                             break;
39

```

死存储消除

说明

进行测试，分数有提升

1552 2024-06-19 11:16:02 1318 / 3600  

```

ConstantFolding.cpp M  DSE.cpp U x  DSE.hpp U  ConstantFolding.hpp  main.cpp M  task4.21305399-周天羽
task > 4 > DSE.cpp > run(llvm::Module& M, llvm::ModuleAnalysisManager& MAM)
6 ~ DeadStorageElimination::run(llvm::Module& M, llvm::ModuleAnalysisManager& MAM)
7 {
8     int removeCount = 0;
9
10    for (llvm::Function& F : M) {
11        for (llvm::BasicBlock& BB : F) {
12            std::set<llvm::Instruction*> instrToErase;
13            for (auto BI = BB.rbegin(); BI != BB.rend(); ++BI) {
14                llvm::Instruction& I = *BI;
15                if (llvm::isa<llvm::AllocaInst>(I)) {
16                    auto* alloc = llvm::cast<llvm::AllocaInst>(&I);
17                    if (alloc->use_empty())
18                        instrToErase.insert(&I);
19                } else if (llvm::isa<llvm::StoreInst>(I)) {
20                    auto* store = llvm::cast<llvm::StoreInst>(&I);
21                    if (store->getPointerOperand()->hasNUsesOrMore(2) ||
22                        llvm::isa<llvm::GetElementPtrInst>(store->getPointerOperand()))
23                        continue;
24
25                    instrToErase.insert(&I);
26                    auto* toDeleteValueUse = store->getValueOperand();
27                    auto* toDeleteDesUse = store->getPointerOperand();
28
29                    std::deque<llvm::Value*> queue;
30                    if (auto* valInstr =
31                        llvm::dyn_cast<llvm::Instruction>(toDeleteValueUse))
32                        if (valInstr->hasOneUse())
33                            if (llvm::isa<llvm::CallInst>(valInstr))
34                                queue.push_back(toDeleteValueUse);
35                            else
36                                queue.push_back(toDeleteValueUse);
37
38                    while (!queue.empty()) {
39                        auto* val = queue.front();
40                        queue.pop_front();
41                        for (auto UI = val->use_begin(), UE = val->use_end(); UI != UE;) {
42                            llvm::Use& U = *UI;
43                            ++UI;
44                            auto* user = llvm::cast<llvm::Instruction>(U.getUser());
45                            if (user->hasNUsesOrMore(2))
46                                continue;
47                            queue.push_back(user);
48                            instrToErase.insert(user);
49                            U.set(nullptr);
50                        }
51                    }
52                }
53            }
54        }
55    }
56    return removeCount;
57 }

```

公共子表达式消除

这个写了但好像不完全正确，所以没应用。


```

[build]
[build] =====
[build] performance/00_bitset1.sysu.c ..... 48.78/100.00
[build] performance/00_bitset2.sysu.c ..... 48.66/100.00
[build] performance/00_bitset3.sysu.c ..... 48.45/100.00
[build] performance/01_mm1.sysu.c ..... 98.30/100.00
[build] performance/01_mm2.sysu.c ..... 100.00/100.00
[build] performance/01_mm3.sysu.c ..... 98.45/100.00
[build] performance/algebraic-identity-1.sysu.c ..... 51.75/100.00
[build] performance/algebraic-identity-2.sysu.c ..... 53.36/100.00
[build] performance/algebraic-identity-3.sysu.c ..... 60.40/100.00
[build] performance/const-propagation-1.sysu.c ..... 5.36/100.00
[build] performance/const-propagation-2.sysu.c ..... 4.70/100.00
[build] performance/const-propagation-3.sysu.c ..... 5.00/100.00
[build] performance/crypto-1.sysu.c ..... 77.22/100.00
[build] performance/crypto-2.sysu.c ..... 75.63/100.00
[build] performance/crypto-3.sysu.c ..... 81.64/100.00
[build] performance/dead-code-elimination-1.sysu.c ..... 100.00/100.00
[build] performance/dead-code-elimination-2.sysu.c ..... 100.00/100.00
[build] performance/dead-code-elimination-3.sysu.c ..... 100.00/100.00
[build] performance/dead-storage-elimination-1.sysu.c ..... 54.77/100.00
[build] performance/dead-storage-elimination-2.sysu.c ..... 18.09/100.00
[build] performance/dead-storage-elimination-3.sysu.c ..... 35.81/100.00
[build] performance/fft0.sysu.c ..... 63.52/100.00
[build] performance/fft1.sysu.c ..... 64.09/100.00
[build] performance/fft2.sysu.c ..... 67.25/100.00
[build] performance/hoist-1.sysu.c ..... 14.54/100.00
[build] performance/hoist-2.sysu.c ..... 14.45/100.00
[build] performance/hoist-3.sysu.c ..... 15.49/100.00
[build] performance/if-combine1.sysu.c ..... 39.46/100.00
[build] performance/if-combine2.sysu.c ..... 39.30/100.00
[build] performance/if-combine3.sysu.c ..... 43.41/100.00
[build] performance/instruction-combining-1.sysu.c ..... 2.92/100.00
[build] performance/instruction-combining-2.sysu.c ..... 2.35/100.00
[build] performance/instruction-combining-3.sysu.c ..... 1.65/100.00
[build] performance/integer-divide-optimization-1.sysu.c ..... 7.70/100.00
[build] performance/integer-divide-optimization-2.sysu.c ..... 5.25/100.00
[build] performance/integer-divide-optimization-3.sysu.c ..... 5.72/100.00
[build]
[build] task4
[build] 总分（加权）：45.93/100.00
[build] =====

```